WEEK 10

Advanced SQL Query System

Scenario:

An e-commerce company wants to analyse its business performance using an E-Commerce Database. The management team needs detailed information about customer purchases, product pricing, and sales performance.

To achieve this, the company uses Subqueries and Nested Queries in SQL to identify premium products, highest-spending customers, best-selling products, and overall sales performance.

These queries help the management team understand customer behaviour, compare product prices, identify profitable categories, and make better business decisions.

Required Tables and Attributes

1. CUSTOMER Table

- CUSTOMER_ID (Primary Key)
- CUSTOMER_NAME
- EMAIL
- CITY

2. ORDERS Table

- ORDER_ID (Primary Key)
- CUSTOMER_ID (Foreign Key)
- ORDER_DATE
- TOTAL_AMOUNT
- ORDER_STATUS

3. PRODUCT Table

- PRODUCT_ID (Primary Key)
- PRODUCT_NAME
- PRICE

- CATEGORY_ID (Foreign Key)

4. CATEGORY Table

- CATEGORY_ID (Primary Key)
- CATEGORY_NAME

5. ORDER_ITEMS Table

- ORDER_ITEM_ID (Primary Key)
- ORDER_ID (Foreign Key)
- PRODUCT_ID (Foreign Key)
- QUANTITY

1. Implement Subqueries and Nested Queries Concept:

A subquery is a SQL query written inside another SQL query, also called an inner query or nested query. The inner query executes first, and its result is used by the outer query. It is used to retrieve data, compare values, filter records and perform complex database operations. The three types are Single-row Subquery, Multi-row Subquery and Nested Subquery.

Find Average Product Price:

This query calculates the average price of all products available in the Product table using the AVG() function.

```
SQL> SELECT AVG(PRICE) FROM PRODUCT;

AVG(PRICE)
----------
     12930
```

Find Products Above Average Price:

This query displays products whose prices are higher than the average price of all products. The inner query calculates the average price, and the outer query retrieves the matching product names and prices.

```
SQL> SELECT PRODUCT_NAME,PRICE FROM PRODUCT WHERE PRICE>(SELECT AVG(PRICE) FROM PRODUCT);

PRODUCT_NAME                                        PRICE
-------------------------------------------------- ----------
Laptop                                              55000
Mobile                                              15000
Tablet                                              25000
```

2. Find Products Above Average Price Task:

The company wants to identify premium and expensive products by comparing product prices with the average price. Subqueries are used to find products above the overall average price and category-wise average price.

1. Find Products Priced Above the Average Price

This query displays products whose prices are greater than the average price of all products. It helps identify premium products.

```
SQL> SELECT * FROM PRODUCT WHERE PRICE > (SELECT AVG(PRICE) FROM PRODUCT);

PRODUCT_ID PRODUCT_NAME                                        PRICE
---------- -------------------------------------------------- ----------
CATEGORY_ID
-----------
       201 Laptop                                             55000
         1

       202 Mobile                                             15000
         2

       206 Tablet                                             25000
         2
```

2. Display Product Name, Category and Price

This query displays the product name, category name and price of products priced above the overall average. It helps management identify expensive products in different categories.

It also helps compare product prices and understand category-wise pricing. This information supports better product pricing strategies and helps the company identify premium products.

```
SQL> SELECT P.PRODUCT_NAME, C.CATEGORY_NAME, P.PRICE FROM PRODUCT P, CATEGORY C WHERE P.CATEGORY_ID = C.CATEGORY_ID;

PRODUCT_NAME
--------------------------------------------------
CATEGORY_NAME                                      PRICE
-------------------------------------------------- ----------
Laptop
Electronics                                         55000

Mobile
Clothing                                            15000

Headphone
Books                                                2000

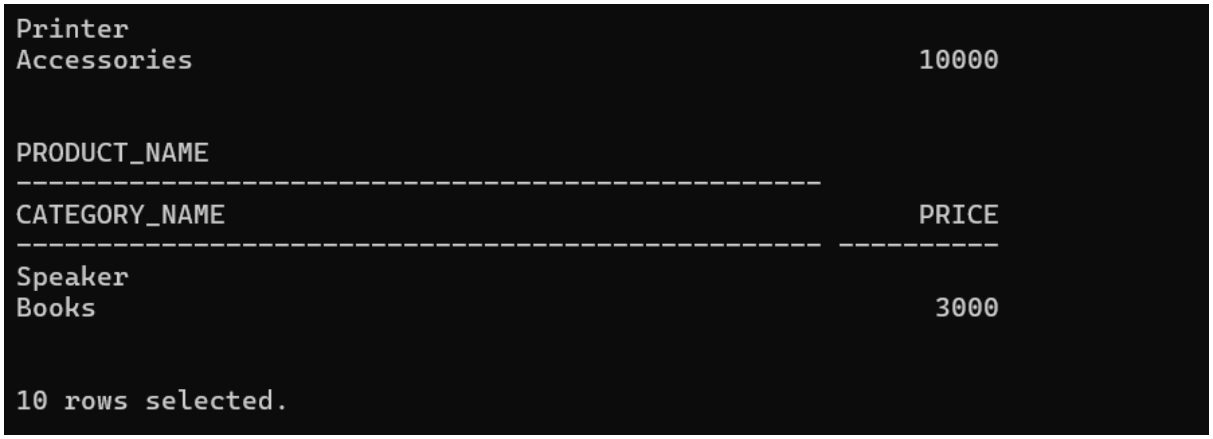

PRODUCT_NAME
--------------------------------------------------
CATEGORY_NAME                                      PRICE
-------------------------------------------------- ----------
Keyboard
Furniture                                            1500

Mouse
Furniture                                             800

Tablet
Clothing                                            25000


PRODUCT_NAME
--------------------------------------------------
CATEGORY_NAME                                      PRICE
-------------------------------------------------- ----------
Monitor
Furniture                                           12000

Smartwatch
Books                                                5000
```



3. Identify Expensive Products in Each Category:

This query identifies the highest-priced product in each category. The subquery calculates the maximum price for each category**.**

The outer query displays the product name, category ID and price of the most expensive products. It helps management compare product prices across different categories and identify premium products.

```
SQL> SELECT * FROM PRODUCT P WHERE PRICE = ( SELECT MAX(PRICE) FROM PRODUCT
WHERE CATEGORY_ID = P.CATEGORY_ID);

PRODUCT_ID PRODUCT_NAME                                            PRICE
---------- ---------------------------------------------------- ----------
CATEGORY_ID
-----------
       201 Laptop                                                55000
         1

       206 Tablet                                                25000
         2

       207 Monitor                                               12000
         4


PRODUCT_ID PRODUCT_NAME                                            PRICE
---------- ---------------------------------------------------- ----------
CATEGORY_ID
-----------
       208 Smartwatch                                             5000
         3

       209 Printer                                               10000
         5
```

4. **Find Products with Price Higher Than a Selected Category Average:**

This query displays products whose prices are higher than the average price of category ID 1. It helps compare product prices within a selected category.

Change CATEGORY_ID = 1 to another category ID when required.

```
SQL> SELECT * FROM PRODUCT WHERE CATEGORY_ID = 4 AND PRICE > (SELECT AVG(PRI
CE) FROM PRODUCT  WHERE CATEGORY_ID = 4);

PRODUCT_ID PRODUCT_NAME                                            PRICE
---------- ---------------------------------------------------- ----------
CATEGORY_ID
-----------
       207 Monitor                                               12000
         4
```

3. **Identify Customers with Maximum Purchases**:

The company wants to analyse customer purchasing behaviour using SQL queries. These queries help identify customers who spend the highest amount, place more orders and contribute more revenue to the business. This analysis helps management understand customer value and improve customer relationships.

1. **Customer Who Spent the Highest Amount**

This query identifies the customer who has spent the highest total amount on purchases. The inner query calculates the total spending of each customer, and the outer query finds the maximum spending amount. It helps the company identify its most valuable customer.

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID O
RDER BY SUM(TOTAL_AMOUNT) DESC;

CUSTOMER_ID SUM(TOTAL_AMOUNT)
----------- -----------------
          1             55000
          5             40000
          9             35000
          2             30000
         10             28000
          4             25000
          8             22000
          7             18000
          3             15000
          6             12000

10 rows selected.
```

2. **Customers with Maximum Number of Orders:**

This query identifies customers who have placed the maximum number of orders. The inner query counts the orders for each customer, and the outer query finds the highest order count. It helps understand customer purchasing frequency.

It also helps the company identify regular customers and analyse their shopping habits. This information supports better customer service and marketing strategies.

```
SQL> SELECT CUSTOMER_ID, COUNT(ORDER_ID) AS TOTAL_ORDERS FROM ORDERS GROUP B
Y CUSTOMER_ID HAVING COUNT(ORDER_ID) = (SELECT MAX(ORDER_COUNT) FROM ( SELEC
T COUNT(ORDER_ID) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOMER_ID));

CUSTOMER_ID TOTAL_ORDERS
----------- ------------
          1            1
          6            1
          2            1
          4            1
          5            1
          8            1
          3            1
          7            1
          9            1
         10            1

10 rows selected.
```

3. Customers whose spending is above average:

This query identifies customers whose total spending is above the average spending.

The SUM() function calculates the total purchase amount of each customer.

The GROUP BY clause groups orders based on Customer ID.

The subquery calculates the average spending of all customers.

The HAVING clause displays customers whose spending is greater than the average.

```
SQL> SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING FROM ORDERS GRO
UP BY CUSTOMER_ID HAVING SUM(TOTAL_AMOUNT) >(SELECT AVG(CUSTOMER_TOTAL) FROM
 (SELECT SUM(TOTAL_AMOUNT) AS CUSTOMER_TOTAL FROM ORDERS GROUP BY CUSTOMER_I
D));

CUSTOMER_ID TOTAL_SPENDING
----------- --------------
          1          55000
          2          30000
          5          40000
          9          35000
```

4. Top 5 valuable customers.

This query identifies the top 5 valuable customers based on their total spending.

The SUM() function calculates the total purchase amount of each customer.

The GROUP BY clause groups orders based on Customer ID.

The ORDER BY clause sorts customers in descending order of total spending.

The FETCH FIRST 5 ROWS ONLY clause displays the top 5 customers with the highest spending.

```
SQL> SELECT CUSTOMER_ID, TOTAL_SPENDING FROM (SELECT CUSTOMER_ID, SUM(TOTAL_
AMOUNT) AS TOTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID ORDER BY TOTAL_SP
ENDING DESC ) WHERE ROWNUM <= 5;

CUSTOMER_ID TOTAL_SPENDING
----------- --------------
          1          55000
          5          40000
          9          35000
          2          30000
         10          28000
```

4.Generate Complex Business Queries

Query 1: Best-Selling Product

- This query identifies the best-selling products based on the total quantity sold.
- The SUM() function calculates the total quantity sold for each product.
- The GROUP BY clause groups sales records based on Product ID and Product Name.
- The SUM() function also calculates the total revenue generated from each product.
- The ORDER BY DESC clause displays products from highest to lowest sales quantity.

```
SQL> SELECT P.PRODUCT_NAME, SUM(OI.QUANTITY) AS TOTAL_QUANTITY_SOLD, SUM(OI.
QUANTITY * P.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_ITEMS OI ON P
.PRODUCT_ID = OI.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY T
OTAL_QUANTITY_SOLD DESC;

PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Mouse                                                                4
         3200

Headphone                                                            3
         6000

Smartwatch                                                           3
        15000


PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Mobile                                                               2
        30000

Speaker                                                              2
         6000

Keyboard                                                             2
         3000
```

```
PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Monitor                                                              2
        24000

Laptop                                                               1
        55000

Tablet                                                               1
        25000


PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Printer                                                              1
        10000


10 rows selected.
```

Query 2: High-Value Customers

Find customers whose total purchase amount is greater than average customer spending.

- This query identifies customers whose total purchase amount is greater than the average customer spending.
- The SUM() function calculates the total purchase amount of each customer.
- The GROUP BY clause groups orders based on Customer ID.
- The subquery calculates the average spending of all customers.
- The HAVING clause displays customers whose total spending is higher than the average spending.

```
SQL> SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING FROM ORDERS GRO
UP BY CUSTOMER_ID HAVING SUM(TOTAL_AMOUNT) > (SELECT AVG(CUSTOMER_TOTAL) FRO
M (SELECT SUM(TOTAL_AMOUNT) AS CUSTOMER_TOTAL FROM ORDERS GROUP BY CUSTOMER_
ID));

CUSTOMER_ID TOTAL_SPENDING
----------- --------------
          1          55000
          2          30000
          5          40000
          9          35000
```

Query 3: Category Performance Analysis

- This query analyzes the sales performance of each product category.
- The JOIN clause combines Category, Product, and Order_Items tables.
- The SUM() function calculates the total revenue generated by each category.
- The COUNT() function calculates the total number of products sold in each category.
- The AVG() function calculates the average product price for each category.

```
SQL> SELECT C.CATEGORY_NAME, SUM(OI.QUANTITY * P.PRICE) AS TOTAL_REVENUE, SU
M(OI.QUANTITY) AS PRODUCTS_SOLD, AVG(P.PRICE) AS AVERAGE_PRICE FROM CATEGORY
 C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN ORDER_ITEMS OI ON P.
PRODUCT_ID = OI.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME ORDER BY
TOTAL_REVENUE DESC;

CATEGORY_NAME                                      TOTAL_REVENUE PRODUCTS_SO
LD
-------------------------------------------------- ------------- -----------
--
AVERAGE_PRICE
-------------
Clothing                                                   55000
 3
        20000

Electronics                                                55000
 1
        55000

Furniture                                                  30200
 8
   4766.66667


CATEGORY_NAME                                      TOTAL_REVENUE PRODUCTS_SO
LD
-------------------------------------------------- ------------- -----------
--
AVERAGE_PRICE
-------------
Books                                                      27000
 8
   3333.33333

Accessories                                                10000
 1
        10000
```

Query 4: Customer Purchase History

- This query displays the purchase history of each customer in the e-commerce database.
- The JOIN clause combines Customer, Orders, Order_Items, and Product tables.
- The COUNT(DISTINCT) function calculates the total number of orders placed by each customer.
- The SUM() function calculates the total purchase amount of each customer.
- The subquery identifies the most purchased product based on the highest quantity ordered by each customer.

```
CUSTOMER_NAME                                      TOTAL_ORDERS
-------------------------------------------------- ------------
TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODUCT
--------------------- --------------------------------------------------
Arun                                                          1
                55000 Mouse

Meena                                                         1
                35000 Mouse

Kaviya                                                        1
                15000 Mouse


CUSTOMER_NAME                                      TOTAL_ORDERS
-------------------------------------------------- ------------
TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODUCT
--------------------- --------------------------------------------------
Priya                                                         1
                30000 Mouse

Rahul                                                         1
                25000 Mouse

Ajay                                                          1
                28000 Mouse
```

```
CUSTOMER_NAME                                      TOTAL_ORDERS
-------------------------------------------------- ------------
TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODUCT
--------------------- --------------------------------------------------
Sneha                                                         1
                18000 Mouse

Vijay                                                         1
                12000 Mouse

Karthik                                                       1
                22000 Mouse


CUSTOMER_NAME                                      TOTAL_ORDERS
-------------------------------------------------- ------------
TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODUCT
--------------------- --------------------------------------------------
Divya                                                         1
                40000 Mouse


10 rows selected.
```

5. Prepare Advanced SQL Report

   Students should generate the following reports:

   Report 1: Premium Product Report

- This report identifies premium products whose price is greater than the average product price.
- The JOIN clause combines Product and Category tables.
- The AVG() function calculates the average price of all products.
- The subquery selects products whose price is above the average price.
- The report displays product name, category, price, and stock availability condition.

```
SQL> SELECT P.PRODUCT_NAME, C.CATEGORY_NAME, P.PRICE FROM PRODUCT P JOIN CATE
GORY C ON P.CATEGORY_ID = C.CATEGORY_ID WHERE P.PRICE > (SELECT AVG(PRICE) FR
OM PRODUCT);

PRODUCT_NAME
--------------------------------------------------
CATEGORY_NAME                                      PRICE
-------------------------------------------------- ----------
Laptop
Electronics                                         55000

Mobile
Clothing                                            15000

Tablet
Clothing                                            25000
```

Report 2: Customer Value Report

- This report displays the purchase details and spending information of each customer.
- The JOIN clause combines Customer and Orders tables.
- The COUNT() function calculates the total number of orders placed by each customer.
- The SUM() function calculates the total spending of each customer.
- The CASE statement classifies customers into Premium and Regular categories based on their total spending.

```
SQL> SELECT C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS, SUM(O.TOTAL_A
MOUNT) AS TOTAL_SPENDING, CASE WHEN SUM(O.TOTAL_AMOUNT) >= 100000 THEN 'Premi
um' ELSE 'Regular' END AS CUSTOMER_CATEGORY FROM CUSTOMER C JOIN ORDERS O ON
C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER B
Y TOTAL_SPENDING DESC;

CUSTOMER_NAME                                    TOTAL_ORDERS TOTAL_SPENDIN
G
-------------------------------------------------- ------------ -------------
-
CUSTOME
-------
Arun                                                        1          5500
0
Regular

Divya                                                       1          4000
0
Regular

Meena                                                       1          3500
0
Regular
```

```
CUSTOMER_NAME                                      TOTAL_ORDERS TOTAL_SPENDIN
G
-------------------------------------------------- ------------ -------------
-
CUSTOME
-------
Priya                                                         1          3000
0
Regular

Ajay                                                          1          2800
0
Regular

Rahul                                                         1          2500
0
Regular


CUSTOMER_NAME                                      TOTAL_ORDERS TOTAL_SPENDIN
G
-------------------------------------------------- ------------ -------------
-
CUSTOME
-------
Karthik                                                       1          2200
0
Regular

Sneha                                                         1          1800
0
Regular

Kaviya                                                        1          1500
0
Regular
```

```
CUSTOMER_NAME                                      TOTAL_ORDERS TOTAL_SPENDIN
G
-------------------------------------------------- ------------ -------------
-
CUSTOME
-------
Vijay                                                         1          1200
0
Regular


10 rows selected.
```

Report 3: Sales Performance Report

- This report analyzes the sales performance of products and categories in the e-commerce database.
- The JOIN clause combines Product, Category, and Order_Items tables.
- The SUM() function calculates the total quantity sold for each product.
- The revenue is calculated by multiplying product price by the quantity sold.
- The ORDER BY DESC clause identifies top-selling products based on the highest sales quantity.

High-performing products:

- This query displays the sales performance of each product.
- The JOIN operation combines Product and Order_Items tables.
- The SUM() function calculates the total quantity sold for each product.
- Total revenue is calculated by multiplying quantity by product price.
- The ORDER BY clause displays products based on highest revenue.

```
SQL> SELECT P.PRODUCT_NAME, SUM(OI.QUANTITY) AS TOTAL_QUANTITY_SOLD, SUM(OI.Q
UANTITY * P.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_ITEMS OI ON P.P
RODUCT_ID = OI.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TOTA
L_QUANTITY_SOLD DESC;

PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Mouse                                                                4
         3200

Headphone                                                            3
         6000

Smartwatch                                                           3
        15000
```

```
PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Mobile                                                               2
        30000

Speaker                                                              2
         6000

Keyboard                                                             2
         3000
```

```
PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Monitor                                                              2
        24000

Laptop                                                               1
        55000

Tablet                                                               1
        25000


PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Printer                                                              1
        10000


10 rows selected.
```

Category-wise revenue

- This query calculates the total revenue generated by each product category.
- The JOIN operation combines Category, Product, and Order_Items tables.
- The QUANTITY * PRICE calculates the revenue from each product.
- The SUM() function calculates the total revenue for each category.
- The ORDER BY clause displays categories from highest to lowest revenue.

```
SQL> SELECT C.CATEGORY_NAME, SUM(OI.QUANTITY * P.PRICE) AS TOTAL_REVENUE FROM
 CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN ORDER_ITEMS
OI ON P.PRODUCT_ID = OI.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME OR
DER BY TOTAL_REVENUE DESC;

CATEGORY_NAME                                      TOTAL_REVENUE
-------------------------------------------------- -------------
Clothing                                                   55000
Electronics                                                55000
Furniture                                                  30200
Books                                                      27000
Accessories                                                10000
```

Report 4: Business Decision Report

- This query identifies the top-selling products in the store.
- The JOIN operation combines Product and Order_Items tables.
- The SUM() function calculates the total quantity sold for each product.
- The ORDER BY clause sorts products by highest sales quantity.
- The ROWNUM condition displays only the top 5 selling products.

```
SQL> SELECT PRODUCT_NAME, TOTAL_QUANTITY_SOLD FROM (SELECT P.PRODUCT_NAME, SU
M(OI.QUANTITY) AS TOTAL_QUANTITY_SOLD FROM PRODUCT P JOIN ORDER_ITEMS OI ON P
.PRODUCT_ID = OI.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TO
TAL_QUANTITY_SOLD DESC) WHERE ROWNUM <= 5;

PRODUCT_NAME                                       TOTAL_QUANTITY_SOLD
-------------------------------------------------- -------------------
Mouse                                                                4
Smartwatch                                                           3
Headphone                                                            3
Keyboard                                                             2
Speaker                                                              2
```

Report 4: Business Decision Report

- This report analyzes the overall business performance of the e-commerce store.
- It identifies high-performing and low-performing products based on sales quantity.
- It identifies high-value customers based on their total spending.
- It analyzes revenue generated by the business.
- The report helps management make better business decisions

1. High-performing Products

- This query identifies high-performing products based on sales quantity.
- The JOIN operation combines Product and Order_Items tables.
- The SUM() function calculates the total quantity sold for each product.
- The subquery calculates the average quantity sold across products.
- The HAVING clause displays products with sales equal to or above average.

```
SQL> SELECT P.PRODUCT_NAME, SUM(OI.QUANTITY) AS TOTAL_SOLD, SUM(OI.QUANTITY *
 P.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_ITEMS OI ON P.PRODUCT_ID
 = OI.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME HAVING SUM(OI.QUANTITY
) >= (SELECT AVG(TOTAL_QTY) FROM (SELECT SUM(QUANTITY) AS TOTAL_QTY FROM ORDE
R_ITEMS GROUP BY PRODUCT_ID)) ORDER BY TOTAL_SOLD DESC;

PRODUCT_NAME                                       TOTAL_SOLD TOTAL_REVENUE
-------------------------------------------------- ---------- -------------
Mouse                                                       4          3200
Headphone                                                   3          6000
Smartwatch                                                  3         15000
```

2. Low-performing Products

- This query identifies low-performing products based on sales quantity.
- The JOIN operation combines Product and Order_Items tables.
- The SUM() function calculates the total quantity sold for each product.
- The subquery calculates the average quantity sold across products.
- The HAVING clause displays products with sales below average.

```
SQL> SELECT P.PRODUCT_NAME, SUM(OI.QUANTITY) AS TOTAL_SOLD, SUM(OI.QUANTITY *
 P.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_ITEMS OI ON P.PRODUCT_ID
 = OI.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME HAVING SUM(OI.QUANTITY
) < (SELECT AVG(TOTAL_QTY) FROM (SELECT SUM(QUANTITY) AS TOTAL_QTY FROM ORDER
_ITEMS GROUP BY PRODUCT_ID)) ORDER BY TOTAL_SOLD ASC;

PRODUCT_NAME                                       TOTAL_SOLD TOTAL_REVENUE
-------------------------------------------------- ---------- -------------
Laptop                                                      1         55000
Printer                                                     1         10000
Tablet                                                      1         25000
Mobile                                                      2         30000
Monitor                                                     2         24000
Speaker                                                     2          6000
Keyboard                                                    2          3000

7 rows selected.
```

3. High-value Customers

- This query identifies high-value customers based on their total spending.
- The JOIN operation combines Customer and Orders tables.
- The SUM() function calculates the total spending of each customer.
- The subquery calculates the average spending of all customers.
- The HAVING clause displays customers whose spending is above average.

```
SQL> SELECT C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTO
MER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,
C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) > (SELECT AVG(CUSTOMER_TOTAL) FROM
 (SELECT SUM(TOTAL_AMOUNT) AS CUSTOMER_TOTAL FROM ORDERS GROUP BY CUSTOMER_ID
)) ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME                                      TOTAL_SPENDING
-------------------------------------------------- --------------
Arun                                                        55000
Divya                                                       40000
Meena                                                       35000
Priya                                                       30000
```

4. Revenue Trends

- This query analyzes the monthly revenue trends of the business.
- The TO_CHAR() function converts the order date into year and month format.
- The SUM() function calculates the total revenue for each month.
- The GROUP BY clause groups orders based on month and year.
- The ORDER BY clause displays revenue trends in chronological order.

```
SQL> SELECT TO_CHAR(ORDER_DATE, 'YYYY-MM') AS REVENUE_MONTH, SUM(TOTAL_AMOUNT
) AS TOTAL_REVENUE FROM ORDERS GROUP BY TO_CHAR(ORDER_DATE, 'YYYY-MM') ORDER
BY REVENUE_MONTH;

REVENUE TOTAL_REVENUE
------- -------------
2026-10        280000
```

Conclusion

The Advanced SQL Query System successfully analyzes the e-commerce database. It identifies high-performing and low-performing products, high-value customers, and monthly revenue trends. Various SQL concepts such as JOIN, GROUP BY, HAVING, subqueries, and aggregate functions are used to generate meaningful business reports. These reports help management understand sales performance, improve customer relationships, and make better business decisions.